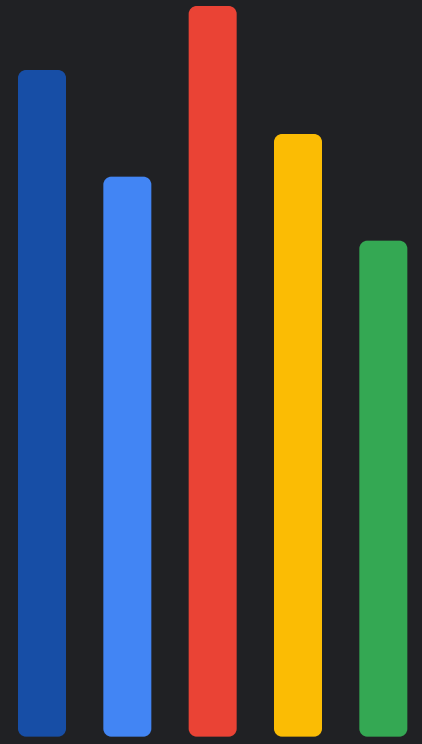
DRAFT PROPOSAL · W3C AUDIO COMMUNITY GROUP

Web Tempo Sync

`navigator.tempoSync`

Promoting the web to a first-class platform for music production and education



Jordan Bayles · Google Chrome

30 October 2026 · TPAC 2026

baylesj.github.io/web-tempo-sync-demo

Music software keeps time together.

Web pages can’t join in.

Ableton Link keeps tempo, bar phase and start/stop in step across Live, Bitwig, Traktor, Rekordbox, Serato, Push, Move, MPCs and hundreds of mobile apps, with no master clock and no setup. Joining takes UDP multicast on the local network, which no web page can do.

NATIVE & HARDWARE ECOSYSTEM

Shared musical timeline on the local network

• Peer-to-peer UDP multicast discovers sessions on Wi-Fi or LAN with zero configuration.
• Every participant shares tempo, bar phase, and transport start/stop without a master clock.

WEB PLATFORM GAP

A web instrument always plays alone

• Web pages cannot open UDP multicast sockets, for good privacy and security reasons.
• Browser-mediated sync bridges the timeline while keeping raw network packets out of the page.

Who needs it

01 · EDUCATION

Music education

A lesson on learn.ableton.com that plays along with the student's Live set or Push, with nothing to install.

02 · LIVE PERFORMANCE

Live coding and visuals

Strudel has wanted Link since 2023. Hydra visuals that land on the beat.

03 · MOBILE WEB

A phone in the room

Open a URL on the same Wi-Fi and play in time with whatever is already playing.

04 · PRODUCTION

Web DAWs and instruments

Lock a browser instrument or sequencer to the gear on stage or in the studio.

Why not an extension? We built one.

TIMING: FINE

~0.1 ms

Clock offset error (0.3 ms best round trip)

Extension plus a native helper gives stable Chrome navigator.tempoSync today. It plays in time with Live.

REACH: NOT FINE

- **A native install, and an extension that can read every site**
- **No Android, ChromeOS or iOS**
- Misses the cases that matter: a lesson that just works, a phone, a school Chromebook
- Direct Sockets is IWA-only; Timing Object has no local network discovery

This is about reach, not timing.

The API: capture, schedule, commit

```
const session = await navigator.tempoSync.requestSession({ quantum: 4 });
const state = session.captureState(audioCtx); // times in AudioContext time

// Schedule the next downbeat
const bar = Math.ceil(state.beatAtTime(audioCtx.currentTime) / 4) * 4;
playKick(state.timeAtBeat(bar));

// Change the tempo for everyone in the room
state.setTempo(128, audioCtx.currentTime);
session.commitState(state);
```

`requestSession()`

One permission prompt. The browser joins the session.

`captureState()`

A consistent snapshot: tempo, beats, phase, start/stop.

`commitState()`

Share what you changed, and only that.

What the design commits to

Choice	What it means
No protocol in the spec	The spec defines the timeline. Link is how browsers would implement it today.
One bit about the network	hasPeers: is anyone else here? No count, addresses or device names.
Shared bar lines, your own beats	Each page numbers beats its own way; starting a loop waits for the next bar line.
Snapshots, not live properties	Consistent timing maths; serializable to Workers and AudioWorklets.
One participant per browser	However many tabs, and it leaves when nothing is visible or audible.

06 · LIVE DEMO

Play along with Live

Prototype Chromium v0.2 and Ableton Live on one laptop. Sampler, Strudel, Hydra, Tone.js and Chrome Music Lab's Arpeggios, each joining the session on its own.

01 · Sampler | 02 · Strudel | 03 · Hydra | 04 · Tone.js | 05 · Arpeggios

↗ baylesj.github.io/web-tempo-sync-demo

Where it stands

Piece	State
Spec	Bikeshed draft, explainer, security and privacy questionnaire, and a list of expected objections
Chromium prototype	v0.2 behind a flag, macOS on Apple silicon; no permission prompt yet
Extension and helper	Stable Chrome on macOS (Linux from source)
Link implementation	Clean-room, sans-IO Rust; plays with Live
Demos	Five tools; a fallback syncs tabs in any browser

All public on GitHub under the topic `web-tempo-sync`.

Open questions

#1 Public support. Known blocker: we need more than one stakeholder saying so.

#2 Protocol and patents. Will Ableton publish a document a standard can cite?

#3 Other engines. Mozilla and WebKit positions.

#4, #10 Timing. A stated accuracy goal, backed by measurements on desktop and Android.

#5 Permission. Prompt wording; Local Network Access and Android's permission.

#6 Protecting a set. An indicator, a follow-only mode, rate limits?

#8 Workers. Should a Worker request a session itself?

#9 quantum. Is Link's model clear to developers new to it?

Numbers are issues on **`github.com/baylesj/web-tempo-sync`**

What I'm asking the group

01 Is this in scope for the Audio CG, and would the group take it on?

02 Would you use it? Say so on issue #1.

03 Weigh in on the open questions, or try the demo and file what breaks.

LIVE DEMO & EXTENSION
baylesj.github.io/web-tempo-sync-demo

DRAFT SPECIFICATION
baylesj.github.io/web-tempo-sync

OPEN ISSUES & FEEDBACK
github.com/baylesj/web-tempo-sync/issues